Database Systems

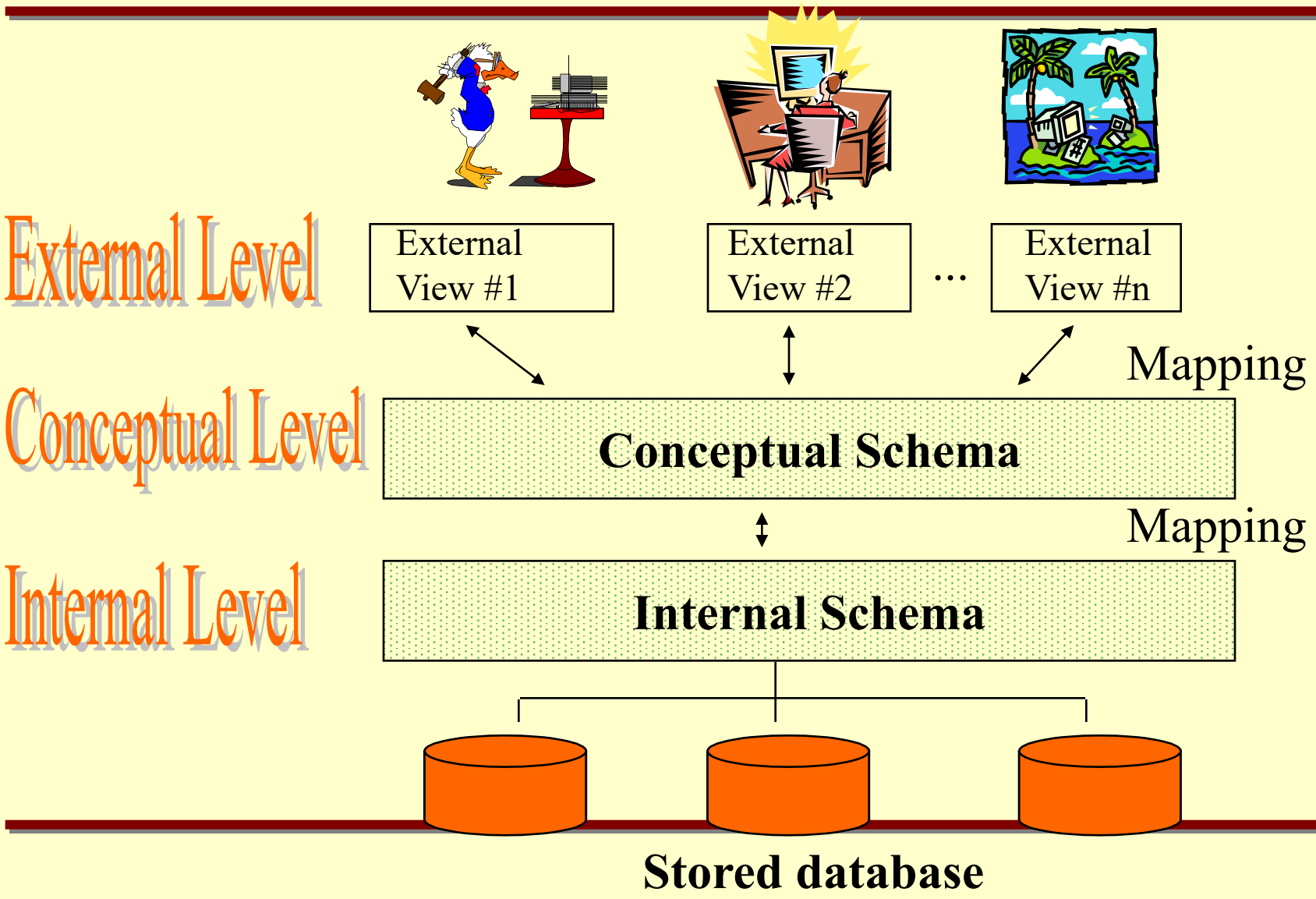
Lecture – 2: RELATIONAL MODEL

Ali Zidane ElQutaany

Database System Lifecycle

- Requirement Analysis
- DB Design logically
- Physical DB and upload data
- Creating application program

Three-Schema Architecture



Three-Schema Architecture

- **Defines DBMS schemas at *three* levels:**
  - **Internal schema** is used to describe physical storage structures and access paths (e.g indexes).
    - Typically uses a physical data model.
  - **Conceptual schema** is used to describe the structure and constraints for the whole database for a community of users.
  - **External schemas** is used to describe the various user views.

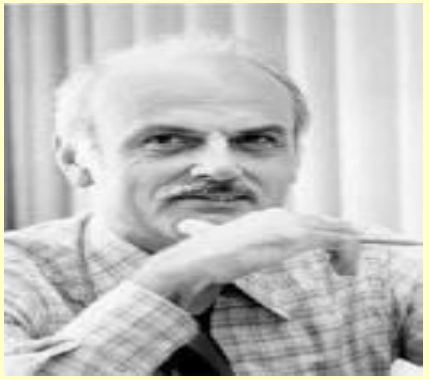
Data Models

Data Models

- A collection of tools for describing:
  - Data
  - Data Relationships
  - Data Semantics
  - Data Constraints
- Models:
  - Relational Model
  - Object-oriented model
  - Semi-structured data models
  - Older models: network model and hierarchical model

History of Relational Model

- Introduced by Ted Codd in 1970 in a classic paper.
- Ted Codd was an IBM Researcher.
- Many database concepts & products based on this model.



Relational Model

Relations

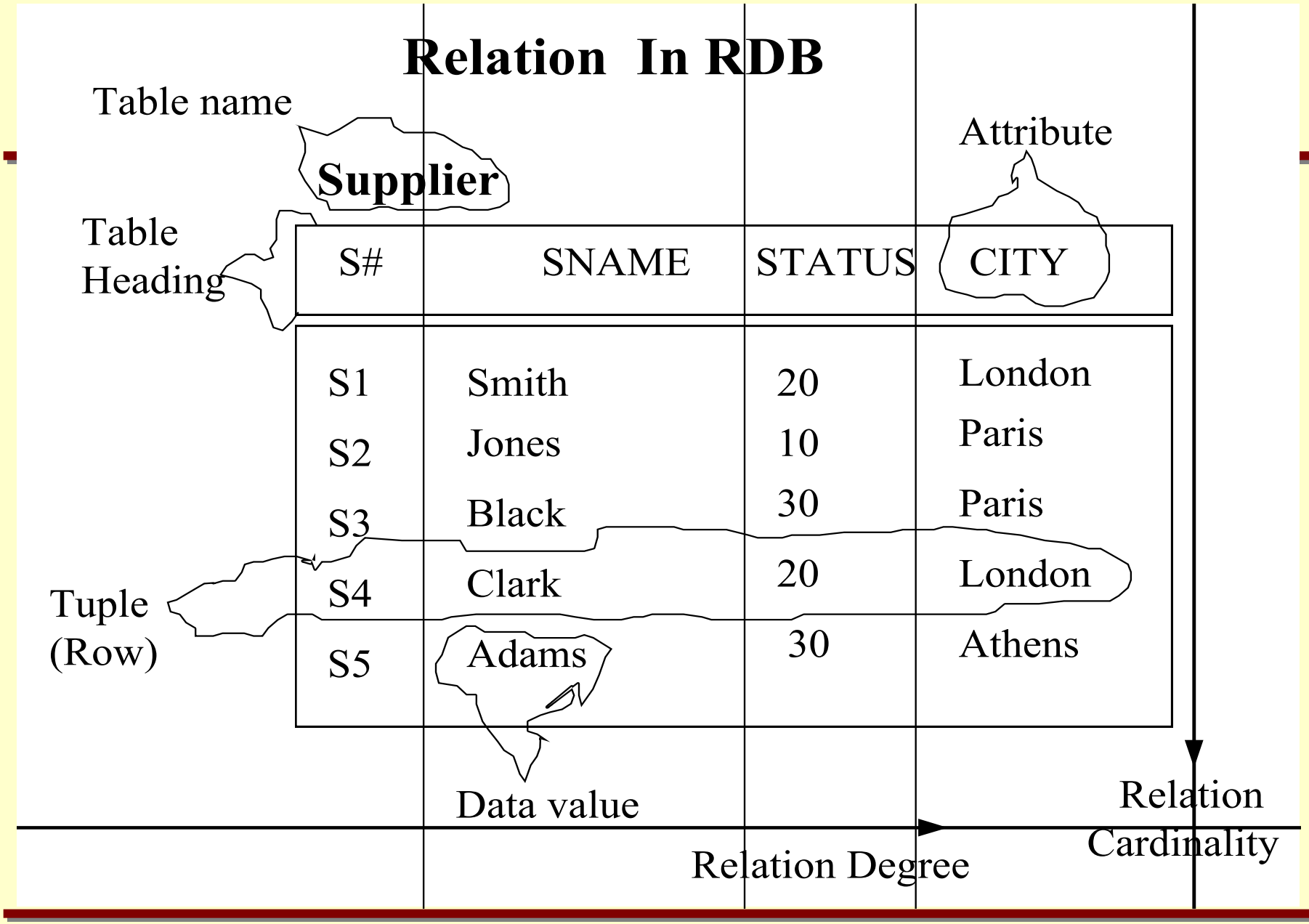
- A relational database is a set of relations.
- Relations are basically **tables** of data.
- Each row represents a **record** in the relation.
- Each relation has a **unique name** in the database.
- Each row in the table specifies a relationship between the values in that row.

acct_id	branch_name	balance
A-301	New York	350
A-307	Seattle	275
A-318	Los Angeles	550
…	…	…

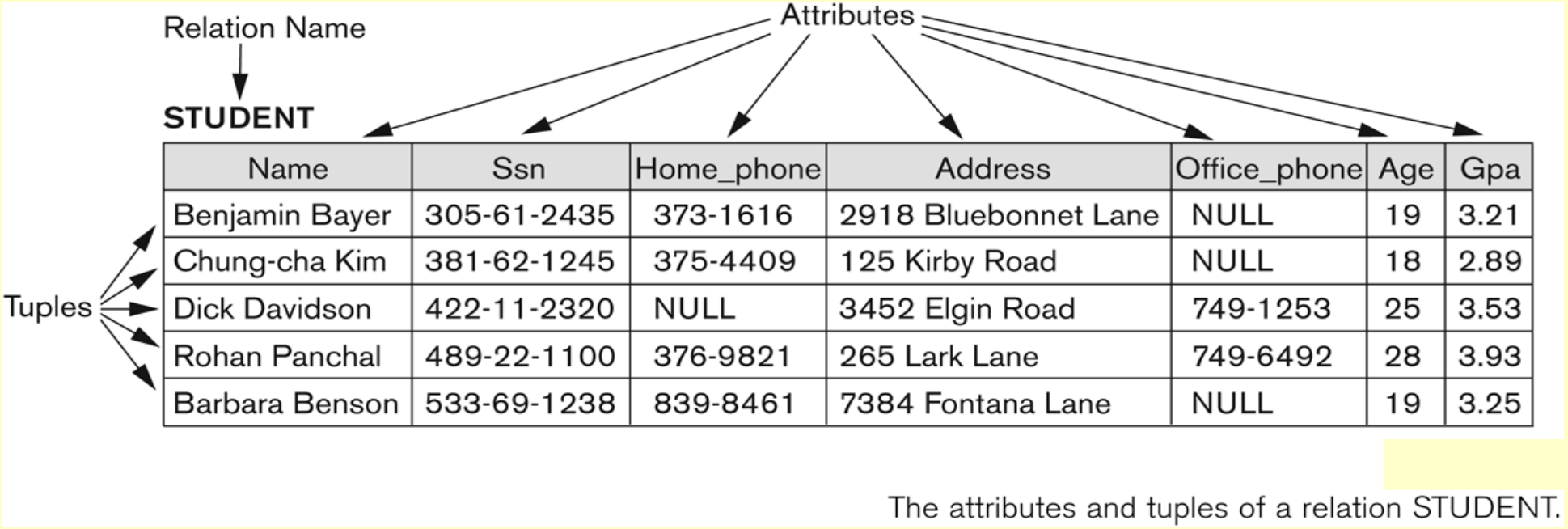
The *account* relation

- Example :

The account ID “A-307”, branch name “Seattle” and balance “275” are all related to each other.


Relation In RDB
Table name
Supplier
Attribute
Table Heading
S#
SNAME
STATUS
CITY
S1
Smith
20
London
S2
Jones
10
Paris
S3
Black
30
Paris
S4
Clark
20
London
S5
Adams
30
Athens
Tuple (Row)
Data value
Relation Cardinality
Relation Degree

Relation Example



Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	749-1253	25	3.53
Rohan Panchal	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	NULL	19	3.25

The attributes and tuples of a relation STUDENT.

Relations and Attributes

- Each relation has some number of attributes.
- Sometimes called "columns".
- Each attribute has a domain specifies the set of valid values for the attribute.

acct_id	branch_name	balance
A-301	New York	350
A-307	Seattle	275
A-318	Los Angeles	550
...	...	...

The *account* relation

- The *account* relation:
  - 3 attributes
  - Domain of *balance* is the set of nonnegative integers
  - Domain of *branch_name* is the set of all valid branch names in the bank

DOMAINS

- A domain D is a set (pool) of values, from which one or more attributes takes their values.
- Example

CITY = {London, Paris, Doha, Cairo, Athens, Rome, Dubai, Madrid}

CITY is a pool of cities from which the attributes Supplier.City, Customer.City take their own values.

DATE = (DAY, MONTH, YEAR)

Where:

DAY = {1..31}, MONTH = {1..12}, YEAR = {1990..2100}

CITY is a simple domain, but DATE is a **composite** Domain

Tuples and Attributes

- Each row is called a tuple
  - A fixed-size, ordered set of name-value pairs
- Each attribute in the tuple has a unique name

acct_id	branch_name	balance
A-301	New York	350
A-307	Seattle	275
A-318	Los Angeles	550
…	…	…

The *account* relation

Tuples and Relations

- A relation is a set of tuples.
- Each tuple appears exactly once.
- The order of tuples in a relation is not relevant.

Schema VS Instance

- The name of the relation and the set of attributes is called the **schema.**
- The current values contained in the relation represent an **instance.**

Relation Schemas

- Every relation has a schema.
- A relation schema includes:
  - An ordered set of attributes
  - The domain of each attribute

acct_id	branch_name	balance
A-301	New York	350
A-307	Seattle	275
A-318	Los Angeles	550
…	…	…

The *account* relation

- The relation schema of *account* is:

Account_schema = (*acct_id*, *branch_name*, *balance*)

COMPANY Database Schema

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date

DEPT_LOCATIONS

Dnumber	Dlocation

PROJECT

Pname	Pnumber	Plocation	Dnum

WORKS_ON

Essn	Pno	Hours

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship

Schema diagram for the COMPANY relational database schema.

Database Schema and Instance

- The **Schema** (or description) of a Relation:
  - Denoted by R(A1, A2, ...An)
  - R is the **name** of the relation
  - The **attributes** of the relation are A1, A2, ..., An
- Example:

  CUSTOMER (Cust-id, Cust-name, Address, Phone#)
- Each attribute has a **domain** or a set of valid values.
  - For example, the domain of Cust-id is 6 digit numbers.
- Instance of R is a set of tuples satisfy the schema of R
- Note
  - **Database schema stable over long period of time**
  - **Database instance change constantly as data inserted or deleted**

Types of DB Constraints

1. Domain constraints
2. Key constraints
3. Integrity constraints
   - Entity Integrity Constraint
   - Referential Integrity Constraint
4. Semantic Integrity Constraints

1- Domains Constraints

- The relational model constrains attribute domains to be atomic
  - Values are indivisible units
- Attribute domains may also include the *null* value
  - *null* = the value is unknown or unspecified
  - *null* can often complicate things. Generally considered good practice to avoid wherever reasonable to do so.

Domain Constraints

Example: “Mobile number” are the set of 11 digits phone numbers valid in Egypt .

Example: Departments of a students are the set of faculty department names

2- Key Constraints

- Value of a key uniquely identifies a tuple in a relation

- A **key** is number of attributes which we cannot remove any attributes and still be able to uniquely identify tuples in a relation

- A relational schema may have more than one key
  - Each key called a ***candidate key***
  - One designated as the ***primary key***

Examples from Premier Database – Primary Key

Customer

CustomerNum	CustomerName	Street	City	State	Zip	Balance	CreditLimit	RepNum
148	Al's Appliance and Sport	2837 Greenway	Fillmore	FL	33336	$6,550.00	$7,500.00	20
282	Brookings Direct	3827 Devon	Grove	FL	33321	$431.50	$10,000.00	35
356	Ferguson's	382 Wildwood	Northfield	FL	33146	$5,785.00	$7,500.00	65
408	The Everything Shop	1828 Raven	Crystal	FL	33503	$5,285.25	$5,000.00	35
462	Bargains Galore	3829 Central	Grove	FL	33321	$3,412.00	$10,000.00	65
524	Kline's	838 Ridgeland	Fillmore	FL	33336	$12,762.00	$15,000.00	20
608	Johnson's Department Store	372 Oxford	Sheldon	FL	33553	$2,106.00	$10,000.00	65
687	Lee's Sport and Appliance	282 Evergreen	Altonville	FL	32543	$2,851.00	$5,000.00	35
725	Deerfield's Four Seasons	282 Columbia	Sheldon	FL	33553	$248.00	$7,500.00	35
842	All Season	28 Lakeview	Grove	FL	33321	$8,221.00	$7,500.00	20

CustomerNum uniquely identifies the *Customer* table and is the primary key of this table.

Examples from Premier Database – Primary Key

OrderLine

OrderNum	PartNum	NumOrdered	QuotedPrice
21608	AT94	11	$21.95
21610	DR93	1	$495.00
21610	DW11	1	$399.99
21613	KL62	4	$329.95
21614	KT03	2	$595.00
21617	BV06	2	$794.95
21617	CD52	4	$150.00
21619	DR93	1	$495.00
21623	KV29	2	$1,290.00

OrderNum and **PartNum** make up the primary key Of the OrderLine table. This is what is known as a **Composite Primary key**, that is, primary key that is made up of more than one field.

Other keys

Value of a key uniquely identifies a tuple in a relation

- **Candidate key:** any subset of attributes that can be a primary key.
- The **primary key** is stable and minimum.
- A **super key** K is subset of attributes of **R**

  such that:

– No 2 tuples have same values for K

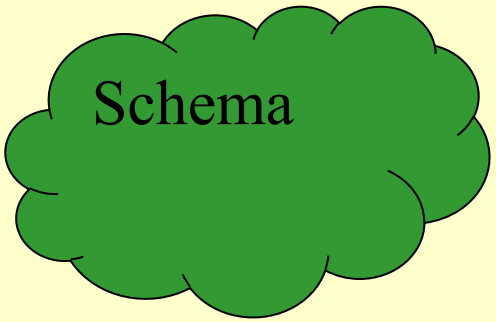
Foreign Keys

- A ***foreign key*** in **R** is a set of attributes **FK** in **R** such that **FK** is a primary key of some other relation **R'**
- A **foreign key** is used to specify a referential integrity constraint

Example

Employee

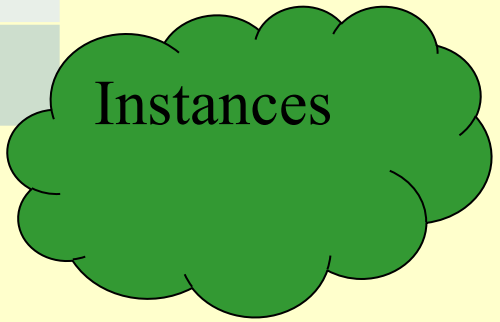
Project



Example

Employee

Enum	Ename	phone	Projectnum
123	Ahmed	01110025878	111
124	Ali	01225929785	
127	Ola	0102457896	111



Project

Pnum	Pname	Location
111	ABC	Giza
112	EFG	Cairo

Example of composite FK

Employee

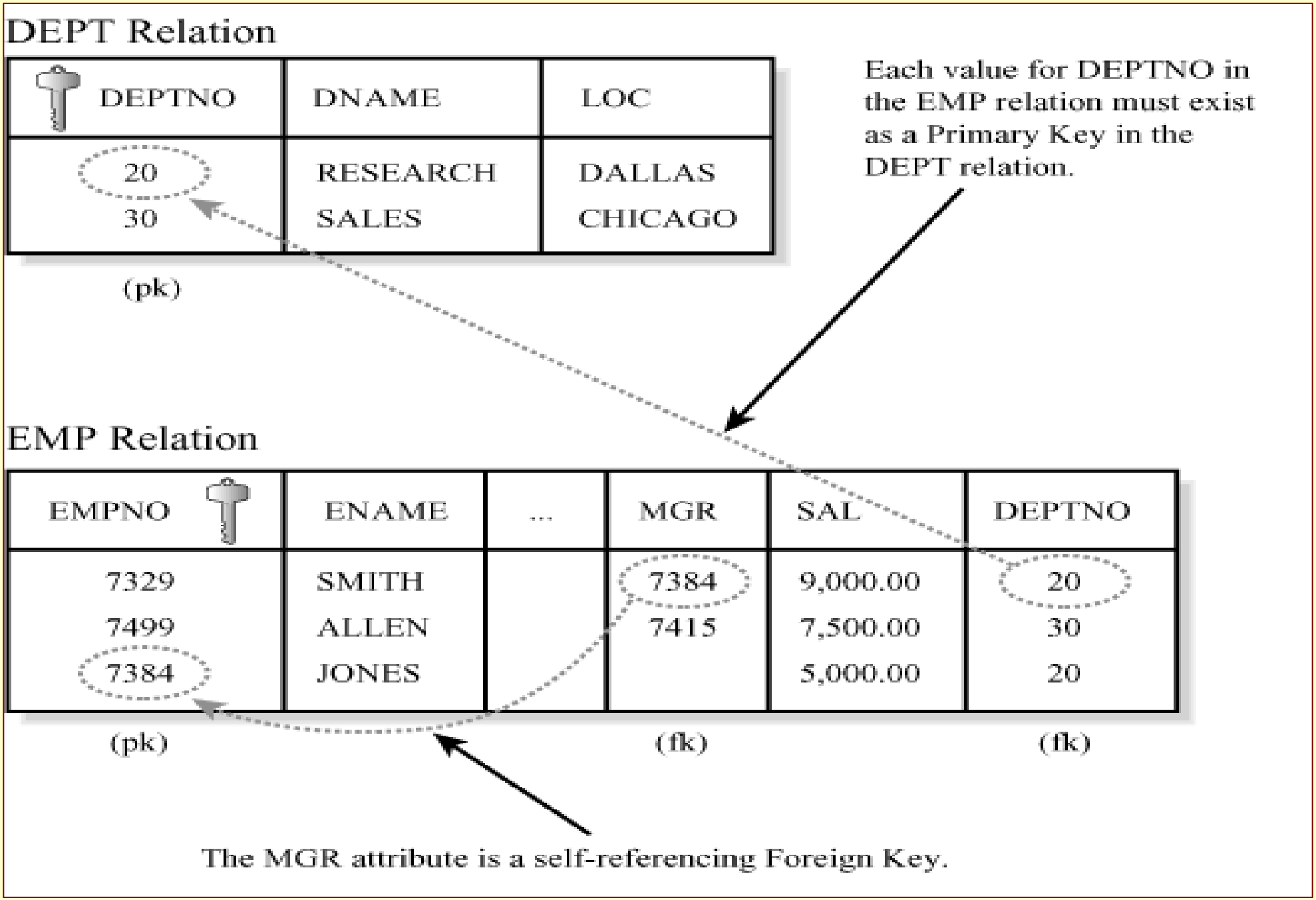
Enum	Ename	phone	Pname	Location
123	Ahmed	01110025878	ABC	Giza
124	Ali	01225929785		
127	Ola	0102457896	EFG	Cairo

Project

Pname	Location
ABC	Giza
EFG	Cairo

Key Constraints

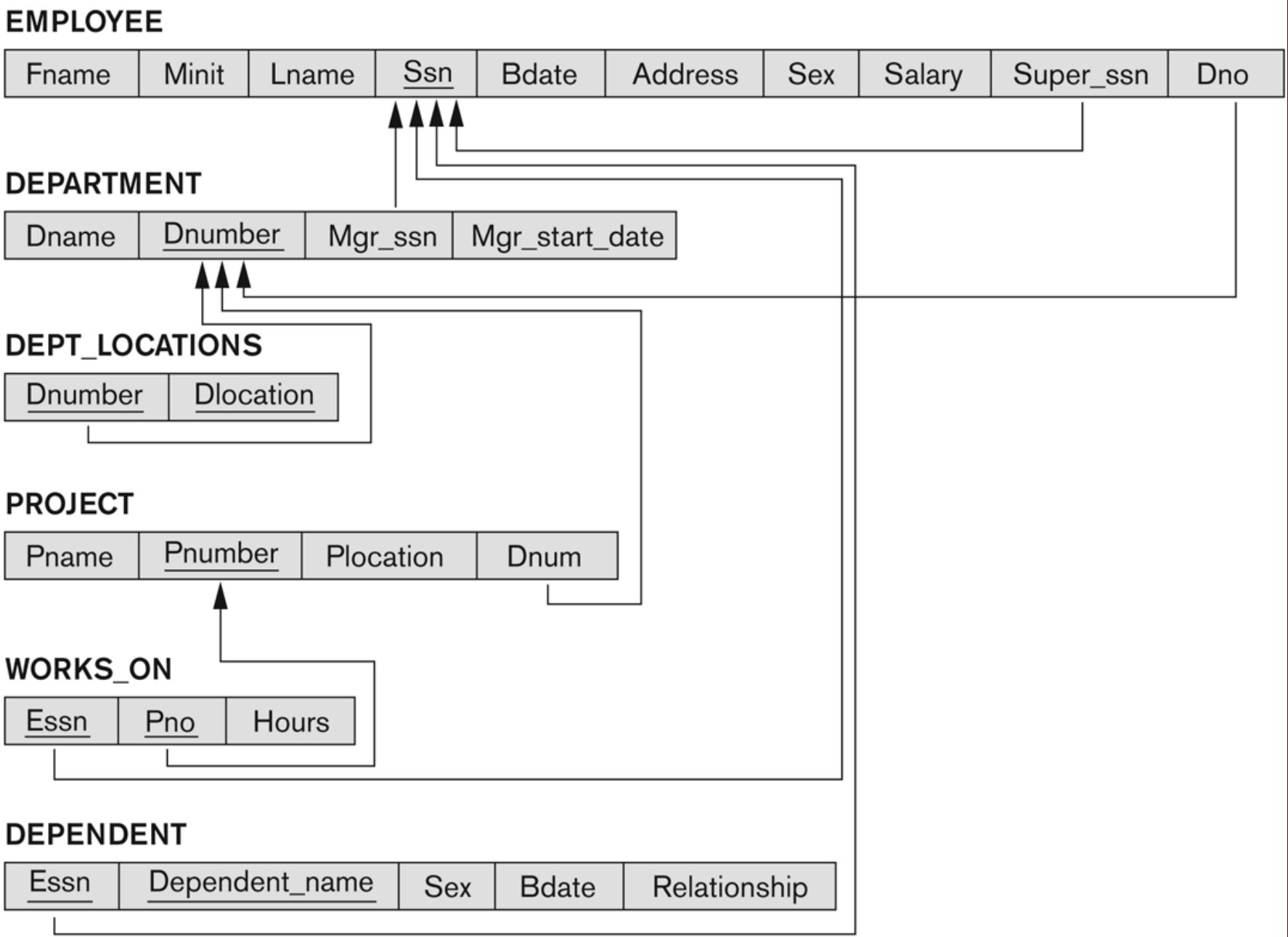
- The two important keys are
  - Primary key
    - Unique
    - Not null
    - **Stable and minimum # of attributes**
  - Foreign key
    - Matches a primary key
    - Can accept Null


DEPT Relation
DEPTNO
DNAME
LOC
20
30
RESEARCH
SALES
DALLAS
CHICAGO
(pk)
Each value for DEPTNO in the EMP relation must exist as a Primary Key in the DEPT relation.
EMP Relation
EMPNO
ENAME
...
MGR
SAL
DEPTNO
7329
7499
7384
SMITH
ALLEN
JONES
7384
7415
9,000.00
7,500.00
5,000.00
20
30
20
(pk)
(fk)
(fk)
The MGR attribute is a self-referencing Foreign Key.

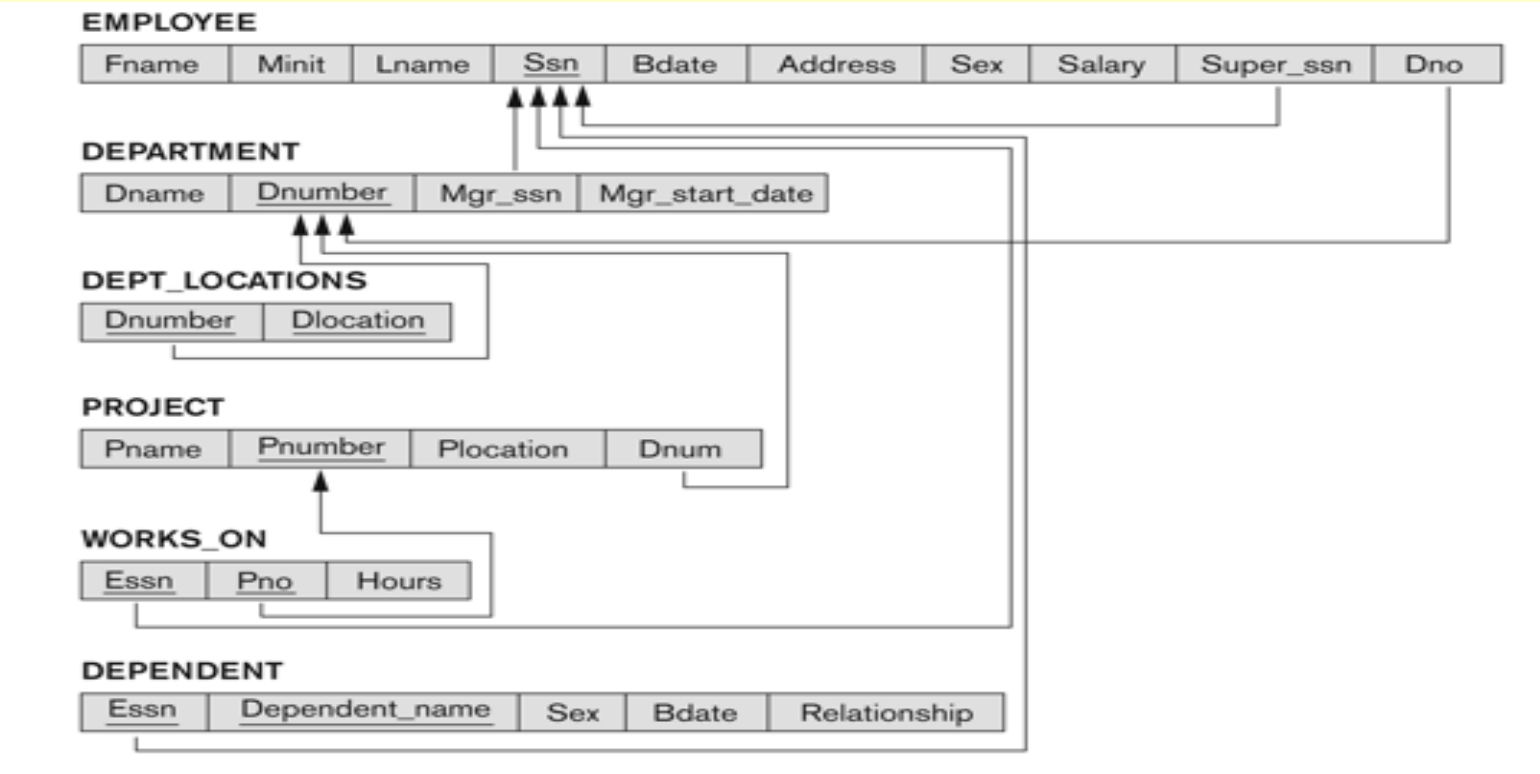
Key Constraints

- Example: Consider the CAR relation schema:
  - **CAR(State, Reg#, SerialNo, Make, Model, Year)**
  - CAR has two candidate keys:
    - Key1 = {State, Reg#}
    - Key2 = {SerialNo}
  - {SerialNo, Make} is a super key.
- **In general:**
  - **Any *key* is a *super key* (but not vice versa)**
  - **Any set of attributes that *includes a key* is a *super key***
  - **A *minimal* super key is also a primary key**

Referential integrity constraints displayed on the COMPANY relational database schema.



Company Relational Schema Diagram



- The primary key attribute (or attributes) will be underlined
- A foreign key (referential integrity) constraints is displayed as a directed arc (arrow) from the foreign key attributes to the referenced table